



YISHUN INNOVA JUNIOR COLLEGE
JC 1 PROMOTIONAL EXAMINATION
Higher 2

CANDIDATE
NAME

CG

INDEX NO

COMPUTING

Paper 1 Written **SOLUTION**

9569/01

28 Sep 2023

1 hour 10 minutes

Additional Materials: Writing Paper
 Cover Page

READ THESE INSTRUCTIONS FIRST

Write your name, index number and class clearly on the cover page.
Write in dark blue or black pen on the writing paper provided.
You may use an HB pencil for any diagrams, graphs, tables or rough working.
Do not use staples, paper clips, glue or correction fluid.

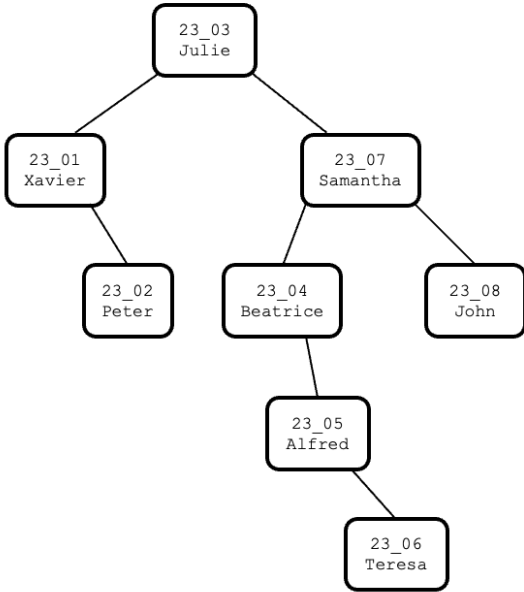
Answer **all** questions.
Approved calculators are allowed.

At the end of the examination, fasten your work securely together.
The number of marks is given in brackets [] at the end of each question or part question.
The total number of marks for this paper is **40**.

1(a)	12 hexadecimal bits = 12 x 4 bits = 48 bits 48 bits = 6 bytes	1m
	Marker's comments: Not well done. Many students were confused between bytes and bits, and gave their answers as 48 bytes.	
(b)	1F:A3:98 1 F A 3 9 8 0001 1111 1010 0011 1001 1000	1m identify first 3 bytes 1m correct conversion
	Marker's comments: Main issue here was not being able to identify the first 6 bytes of the MAC. There were a few cases of students proceeding to convert the hex number to denary.	
(c)	3 bytes = 24 bits $2^{24} = 16\,777\,216$	1m 1m
	Marker's comments: Not well done. For students who did not identify the first 6 bytes correctly in the prev part, ECF was applied here. Generally, students were unable to calculate the maximum range of the number.	

2(a)	For a size n array, we will do (n-1) passes. For the first pass, we compare adjacent element in pairs from start to the end. If they are not in order, we swap. This will bring the biggest element to the end of the array. On the second pass, we repeat the same steps to compare adjacent elements until compare the 2nd last element of the array. This will bring the 2nd biggest element to the 2nd last position. The third pass brings the 3rd largest element to the 3rd last position. By repeat these steps until the (n-1)th passes, the whole array is sorted.	1m: compare adjacent elements from start to end, swap if nec 1m: Repeat passes
	Marker's comments: Need to improve on the clarity of explanations for majority of the students. The idea of "adjacent elements" is not clear in some responses. Also, some responses were unclear whether you are repeating the comparisons between adjacent elements or repeating the passes.	
(b)	If there are no swaps in the current pass, it implies that the array is already sorted, and there is no need to carry out subsequent passes if there are any. We can terminate the process early.	1m

	Marker's comments: Quite a number wrote if no swaps == True, that means the array is sorted. This is incomplete, as you need to mention that the process can be terminated earlier / can break the loop.	
(c)	A suitable pivot will be the median of the data sets, so that the array will be divided into two roughly equal subarrays, and this minimises the number of recursive calls.	1m
	Marker's comments: The correct word to use is median . Average is not fully correct, because average refers to mean, which may not appear in the list. Middle/middle element is also not accepted, unless it is the middle element of the sorted array.	
(d)	A: 0 B: lst[pivotIndex] C: element in lst D: QUICKSORT(left) + middle + QUICKSORT(right)	1m: for any 1 1m: for any 2 1m: for all 4
	Marker's comments: Read the question carefully. The question asks to use the first element as the pivot. Many wrote len(lst)//2 for A, which is incorrect. For C, "element in range(lst)" is not accepted, because the "element" used in the pseudocode is the value, not the index.	
(e)	Pivot (542) is at index position 4. [391, 215, 482, 304, 542, 731, 629] OR [304, 215, 391, 482, 542, 731, 629]	1m 1m
	Marker's comments: Same comment as (d), to read the question carefully, as 542 is the pivot. The pseudocode should be relatively easy to implement if you get the pivot correct.	
(f)	Advantage: Quicksort is more efficient than bubble sort, as its order of growth is $n \log(n)$, compared to bubble sort's order of growth, which is n^2 , for worst case scenario. Disadvantage: Non-in-place quicksort means that extra memory space is required, as a new list is created. However, bubble sort does not require the extra memory space.	1m 1m
	Marker's comments: When explaining the advantage of quicksort, which is its efficiency, the order of growth needs to be stated for both sorting algorithms.	

4(a)	 <pre> graph TD 23_03[23_03 Julie] --> 23_01[23_01 Xavier] 23_03 --> 23_07[23_07 Samantha] 23_01 --> 23_02[23_02 Peter] 23_07 --> 23_04[23_04 Beatrice] 23_07 --> 23_08[23_08 John] 23_04 --> 23_05[23_05 Alfred] 23_05 --> 23_06[23_06 Teresa] </pre> Minus 1 mark per wrong entry	3
	Marker's comments: Generally well done. For students who did not get full credit, it was mostly due to careless mistakes. A few students rearranged the given list. Hence the insertion was not done in the right order.	
(b)	1. Starting from the root node, <u>check if its value matches the data to be found.</u> 2. If not, and <u>root node value is bigger than data, go to the left child, and right child otherwise.</u> 3. The left/right child becomes the current node. <u>Repeat steps 1 and 2</u> 4. <u>Stop the process when the current node data matches the data to be found, or when an empty tree is encountered (leaf node)</u>	1 1 1
	Marker's comments: Not well done. The majority of students did not consider the possibility that the root node contains the key. Most students were able to state clearly how to decide to go left or right. These students were able to state that the search stops when the key is found but generally failed to explain how to know if the key is not in the BST (reach a leaf). Some students confused BST with Binary Search. No credit was given. A handful of students answered this as an insertion algo qn. No credit was given.	
(c)	$O(\log n)$ $O(n)$ 1 mark for both correct	1 1
	Marker's comments:	

	Many students did not understand the requirement of the question and went on to explain in long form how BST is faster than linear search. There is no need for that. Common mistake for BST time complexity is stating it as $O(n \log n)$.	
(d)	Starting from the root node, If there is a left tree, visit it first, followed by the root node, then the right tree (if any) left-root-right	1 1
	Marker's comments: Credit was given for stating to visit left branch, then root node, then right branch. It was not easy to explain clearly how to keep visiting the left branch (if avail). Hence, as long as the order of visiting is clear, full credit was awarded.	

5(a)		2
(b)		2
(c)		2
(d)		2